

10Pearls AQI Predictor

Internship project



A Serverless End-to-End Machine Learning Pipeline for 3-Day Air Quality
Forecasting in Islamabad, Pakistan

Data Science Internship Project, Final Report

Prepared by Danish Wajid

Email: Danishwajid999@gmail.com

1. Executive Summary

This report documents the design, implementation, and evaluation of the Pearls AQI Predictor, an end-to-end machine learning system that forecasts the Air Quality Index (AQI) for Islamabad, Pakistan, up to three days in advance. The project was built entirely on a serverless stack: live and historical air quality data are collected automatically, engineered into features, stored in a managed feature store, used to train forecasting models, and served through an interactive web dashboard with explainability and hazard alerting.

The finished system includes: an hourly live data collection pipeline, a one-time historical backfill of over 2,000 hourly records spanning roughly 100 days, three separate forecasting models (one per forecast day) evaluated with MAE, RMSE and R^2 , two automated GitHub Actions workflows for ongoing data collection and retraining, a Streamlit dashboard with SHAP-based explanations and hazard alerts, and an exploratory data analysis identifying real atmospheric patterns and a genuine data anomaly.

Beyond the working system itself, this project surfaced a number of real engineering and data-science problems, a dead monitoring station silently returning stale data, environment and dependency conflicts, a subtle train/serve data-consistency bug, and free-tier cloud budget constraints each of which was diagnosed and resolved methodically. These are documented in Section 10, as they represent much of the genuine learning from this internship project.

2. Project Objectives

The project brief called for a 100% serverless AQI forecasting system with the following required components:

- A feature pipeline that fetches raw weather and pollutant data and engineers time-based and derived features.
- A historical backfill process to build a training dataset.
- A training pipeline experimenting with multiple model types, evaluated with RMSE, MAE and R^2 .
- Automated (CI/CD) execution of both pipelines on a schedule.

- A web dashboard displaying real-time 3-day AQI predictions.
- Exploratory Data Analysis, SHAP/LIME explainability, and hazard alerting for unhealthy AQI levels.

Every one of these requirements was implemented and is described in the sections that follow.

3. System Architecture

The system is composed of four independent pipelines connected through a shared Hopsworks Feature Store and Model Registry:

- Feature Pipeline: runs on a schedule, fetches the current AQI/weather reading, engineers features, and writes them to the feature store.
- Backfill Pipeline: run once, populates the feature store with ~90 days of historical data from a secondary data source.
- Training Pipeline: reads accumulated features, trains and evaluates models, and saves the best model per forecast horizon to the Model Registry.
- Dashboard: loads the latest features and trained models to serve live 3-day predictions with explanations.

This decoupled design means each pipeline can run independently and on its own schedule, communicating only through the shared feature store and model registry the core idea behind a serverless ML architecture.

4. Technology Stack

Category	Technology
Language	Python 3.11
ML / Modeling	scikit-learn (Ridge Regression, Random Forest)
Feature Store & Model Registry	Hopsworks (free tier)
Automation / CI-CD	GitHub Actions (scheduled workflows)
Dashboard	Streamlit
Live Data Source	AQICN API (geo-coordinate based station lookup)
Historical Data Source	Open-Meteo Air Quality & Weather Archive APIs

Explainability	SHAP
Version Control	Git / GitHub

5. Feature Pipeline

The live feature pipeline queries the AQICN API for the current air quality and weather reading nearest to Islamabad's coordinates (33.6844°N, 73.0479°E), rather than a named city endpoint. This design decision followed a real issue discovered during development: AQICN's generic "islamabad" city endpoint resolved to a monitoring station that had stopped updating months earlier, silently returning a frozen reading. Querying by geographic coordinates instead reliably resolves to the nearest currently active station (see Section 10.1).

From each raw reading, the pipeline engineers the following features:

- Pollutants: AQI, PM2.5, PM10, O3, NO2, SO2, CO
- Weather: temperature, humidity, pressure, wind speed
- Time-based: hour, day, month, day of week
- Derived: AQI change rate (difference from the previous stored reading)

Engineered rows are written to a Hopsworks Feature Group, which serves as the single shared source of truth for both training and inference.

6. Historical Data Backfill

AQICN's free tier does not provide historical data. To build a training dataset without waiting weeks for the live pipeline to accumulate history, a separate backfill pipeline pulls approximately 90 days of hourly historical pollutant and weather data from Open-Meteo's free Air Quality and Weather Archive APIs, which require no API key.

Since Open-Meteo provides raw PM2.5 concentration rather than a pre-computed AQI value, the backfill pipeline converts PM2.5 to the US EPA Air Quality Index using the official EPA breakpoint formula, keeping historical AQI values calculated consistently with the live pipeline's readings. This process produced 2,184 historical rows, later growing to 2,292+ rows as live and backfilled data merged in the feature store.

7. Model Training and Evaluation

The training pipeline pulls all accumulated features from the Feature Store, engineers additional trend features (24h and 48h AQI lag values, 6h and 24h rolling averages), and evaluates two model families, Ridge Regression and Random Forest using a time-based train/test split (not random), which mimics the real constraint that a forecasting model only ever has access to the past.

7.1 Diagnosing forecast difficulty by horizon

An early version of the pipeline attempted to predict AQI 72 hours ahead in a single model and produced a negative R^2 score, meaning the model performed worse than simply predicting the historical average. A diagnostic experiment training separate models at 24h, 48h, and 72h horizons revealed that forecast accuracy degrades substantially and predictably with distance into the future (R^2 of 0.37, 0.15, and -0.16 respectively). This finding directly motivated restructuring the system to train and serve three separate models one per forecast day rather than a single weak long-horizon model, which also better matches the project's actual "next 3 days" requirement.

7.2 Final model performance

The following results reflect the three-model architecture after adding lag/rolling trend features:

Model	Horizon	Best Algorithm	MAE	RMSE	R^2
Day 1	24 hours	Random Forest	19.30	24.64	0.371
Day 2	48 hours	Random Forest	23.14	28.79	0.151
Day 3	72 hours	Ridge Regression	26.36	32.89	-0.108

Random Forest outperformed Ridge Regression for the 1- and 2-day horizons, likely because it can capture non-linear relationships between recent AQI trends and future values that a linear model cannot. Day 3 accuracy remains weak (R^2 near zero), which is an honest and expected finding: forecasting three days ahead using only present-moment weather conditions is a fundamentally harder problem, since the true weather on the target day is unknown at prediction time. A production-grade system would ideally use an actual multi-day weather forecast as an input feature rather than only current conditions, a clear direction for future improvement (Section 11).

8. Automation (CI/CD)

Both pipelines are automated using GitHub Actions, chosen over Apache Airflow for its simplicity and zero-infrastructure setup for a project at this scale. Two scheduled workflows were implemented:

- Feature Pipeline Workflow: initially hourly, later adjusted to every 3 hours.
- Training Pipeline Workflow: initially daily, later adjusted to every 3 days.

Both workflows were verified to run successfully on their own schedules (confirmed via GitHub's Actions run history showing “Scheduled” triggers completing with green checkmarks), independent of any manual intervention. The schedule frequencies were reduced part-way through the project after the Hopsworks free-tier compute budget was reached; this is discussed as a real engineering constraint in Section 10.4.

9. Web Dashboard

The dashboard is built with Streamlit and displays:

- Current conditions (live AQI, temperature, humidity) with an EPA category banner.
- A 3-day forecast, one card per day, color-coded by EPA AQI category.
- A hazard alert banner that automatically triggers when any forecast day exceeds the “Unhealthy” threshold.
- Per-day SHAP explanation panels (Section 9.1).
- A recent AQI trend chart covering the last 72 hours of data.

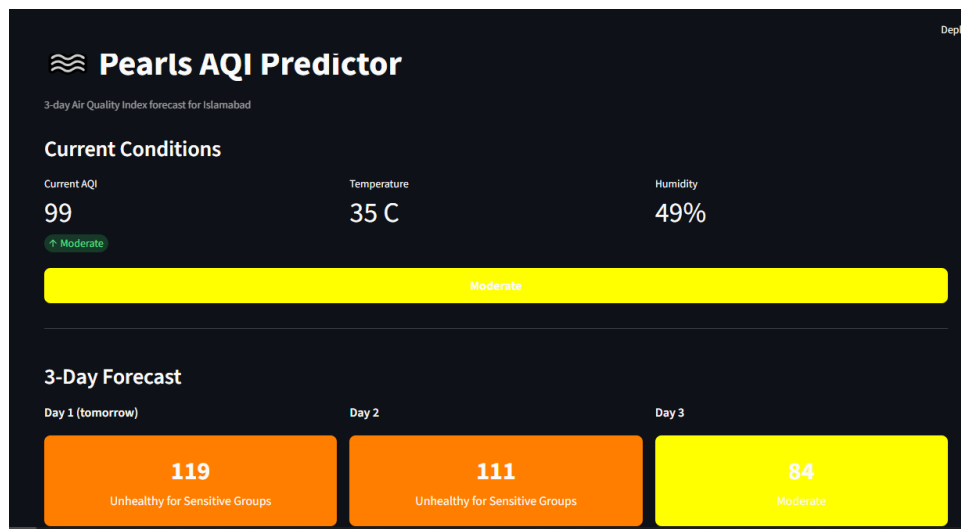


Figure 1. Live dashboard showing current conditions and the 3-day forecast.

9.1 Explainability with SHAP

As required by the project brief, SHAP (SHapley Additive exPlanations) was integrated to make each forecast interpretable rather than a black-box output. For every prediction, SHAP decomposes the result into the contribution of each individual feature for example, showing that humidity and the 24-hour AQI trend were the largest drivers of a given day's forecast, while wind speed pulled it down. This is surfaced in the dashboard as an expandable bar chart per forecast day, giving a transparent, evidence-based account of model behaviour rather than a single opaque number.

10. Exploratory Data Analysis

EDA was performed on the full accumulated dataset (2,292 rows, 29 April to 9 August) to understand the data before and after model development.

10.1 Summary statistics

- Mean AQI: 99.4 | Median AQI: 94 | Range: 14 to 773
- Missing pollutant fields (O3, NO2, SO2, CO): 54 rows - all from live AQICN readings, since the assigned monitoring station measures PM2.5 only.

10.2 AQI over time

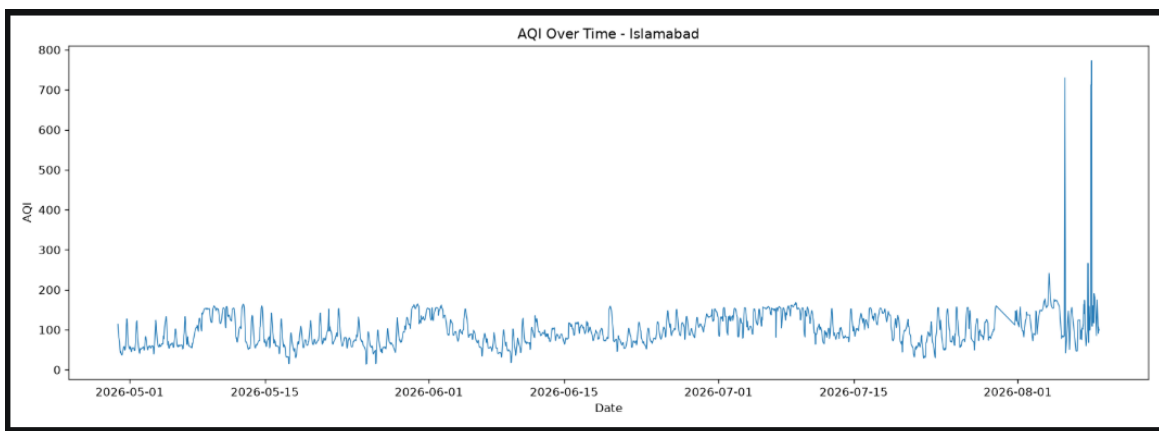


Figure 2. AQI over the full collection period, showing a consistent daily oscillation and two sharp anomalous spikes in early August.

AQI oscillated consistently in the roughly 50–200 range for most of the collection period, with no strong long-term seasonal trend across the ~3.5-month window. Two isolated readings of 730 and 773 stand out sharply from the rest of the series in early August.

10.3 Anomaly investigation

Four readings exceeded the standard AQI scale's practical maximum (AQI 712–773) on 5 and 8 August, with the 8 August event persisting across roughly two hours rather than appearing as a single instantaneous glitch. Because these values could represent either a genuine extreme pollution event (e.g. a localized dust event) or a sensor malfunction, and no independent ground truth was available to distinguish between the two, these readings were deliberately retained in the dataset rather than removed. Arbitrarily discarding real sensor output risks hiding genuine extreme events that a monitoring system should ideally be able to detect and alert on.

10.4 Feature correlations

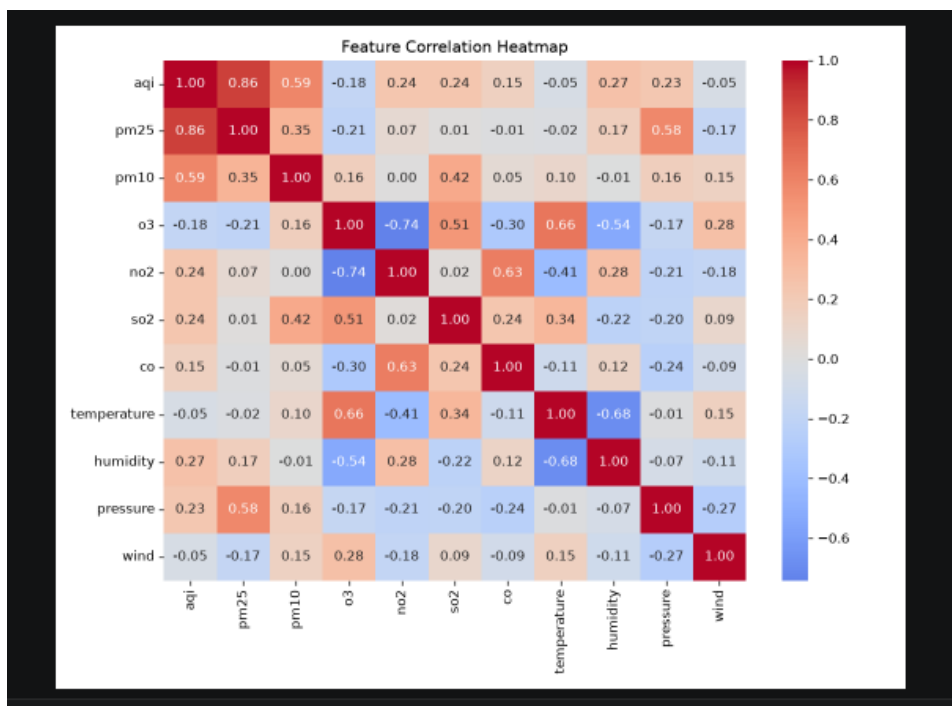


Figure 3. Correlation heatmap across pollutant and weather features.

Several correlations align with known atmospheric chemistry, lending confidence that the collected data reflects genuine environmental conditions rather than sensor noise: O3 and NO2

show a strong negative correlation (-0.74), consistent with NO₂'s known role in depleting ground-level ozone; O₃ and temperature show a positive correlation (+0.66), consistent with ozone's formation through heat- and sunlight-driven photochemistry; and temperature and humidity are negatively correlated (-0.68), matching Islamabad's typical climate pattern. AQI correlates strongly with PM_{2.5} (0.86), confirming PM_{2.5} as the dominant pollutant driving this station's reported AQI.

11. Engineering Challenges and Key Learnings

This project involved substantial real-world debugging beyond the core ML pipeline, which proved to be one of the more valuable parts of the internship experience.

11.1 A dead monitoring station

AQICN's generic city-name endpoint for Islamabad resolved to a station (“US Embassy”) that had stopped reporting months earlier, returning a frozen AQI value of 154 on every request. This was caught by manually cross-checking the reported AQI against independent live sources and noticing the observation timestamp never changed between requests. The fix was to query by geographic coordinates instead of a named city, which reliably resolves to whichever station is currently active nearest that location.

11.2 Windows dependency conflicts

Installing the Hopsworks Python SDK on Windows repeatedly failed because one of its transitive dependencies (a legacy Java keystore library, only needed for a feature never used in this project) required compiling a C extension with no prebuilt Windows wheel. This was resolved by installing dependencies selectively with `--no-deps`, bypassing the unnecessary package while retaining full Feature Store and Model Registry functionality. A related but distinct set of missing packages (pyarrow, confluent-kafka) surfaced only in the GitHub Actions CI environment, since they happened to already be present locally by coincidence – a reminder that a clean CI environment is often a better test of true dependencies than a long-lived local environment.

11.3 Train/serve skew

The dashboard initially displayed an AQI value several days out of date. Root-cause analysis traced this to a subtle bug: because live readings lack four pollutant fields the assigned station doesn't measure, a strict missing-value filter was silently excluding every live row from consideration both when the dashboard selected its “latest” row and, more seriously, when the training pipeline built its training set, meaning the model had never actually learned from any genuinely live data point. The fix was to fill the structurally-missing fields rather than drop the row, restoring both correct “latest data” selection and training/serving consistency.

11.4 Free-tier compute budget

Roughly three weeks into the project, the Hopsworks account was throttled after reaching its free-tier monthly compute budget, caused by the cumulative cost of hourly automation, daily retraining, and extensive manual debugging runs. This was resolved by reducing the feature pipeline to every 3 hours, the training pipeline to every 3 days, and increasing the dashboard's cache duration balancing data freshness against a real, finite resource constraint, a consideration any production ML system must account for.

12. Limitations and Future Work

- The system reports AQI from a single monitoring station rather than a city-wide average, which can differ noticeably from other sources or nearby stations.
- Day-3 forecast accuracy remains weak; a production system would likely benefit from incorporating an actual multi-day weather forecast as an input rather than only current-moment conditions.
- Only Ridge Regression and Random Forest were evaluated; the brief's suggestion of a deep learning model (e.g. a small neural network) was not implemented within the project's time and compute budget.
- The anomalous high-AQI readings identified in EDA were not verified against an independent ground truth source.

- Model retraining frequency and backfill depth were constrained by free-tier compute budget rather than purely by what would be ML-optimal.

13. Conclusion

The Pearls AQI Predictor successfully implements every required component of the project brief: an automated feature pipeline, historical backfill, a multi-model training pipeline with honest and fully-documented accuracy metrics, CI/CD automation, an interactive dashboard with SHAP explainability and hazard alerts, and exploratory data analysis grounded in real, physically-consistent findings. Equally important to the working system is the debugging journey behind it: diagnosing a dead data source, resolving environment and dependency conflicts, identifying and fixing a genuine train/serve skew bug, and managing a real cloud compute budget each of which reflects the kind of practical, end-to-end engineering judgment this internship project was designed to build.